

Received 21 November 2023, accepted 11 December 2023, date of publication 14 December 2023,
date of current version 8 February 2024.

Digital Object Identifier 10.1109/ACCESS.2023.3342840

RESEARCH ARTICLE

Evasion Attacks and Defense Mechanisms for Machine Learning-Based Web Phishing Classifiers

MANU J. PILLAI¹, S. REMYA², V. DEVIKA¹, SOMULA RAMASUBBAREDDY³,
AND YONGYUN CHO³

¹Department of Computer Science and Engineering, Thangal Kunju Musaliar College of Engineering (TKMCE), Kollam, Kerala 691005, India

²School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri Campus, Kollam, Kerala 690525, India

³Department of Information and Communication Engineering, Suncheon National University, Suncheon, Jeollanam-do 57922, South Korea

Corresponding author: Yongyun Cho (yycho@scnu.ac.kr)

This work was supported in part by the Ministry of Science and ICT (MSIT), South Korea, through the Grand Information Technology Research Center Support Program, Supervised by the Institute for Information and Communications Technology Planning and Evaluation (IITP), under Grant IITP-2023-2020-0-01489; in part by the MSIT through the Information Technology Research Center (ITRC) Support Program, Supervised by the IITP, under Grant RS-2023-00259703; and in part by the Interdisciplinary Program in IT-Bio Convergence System (BK21 Plus), Suncheon National University, Suncheon-si, Jeollanam-do, Republic of Korea.

ABSTRACT Phishing is an electronic fraud through which an attacker can access user credentials. Phishing websites are the ones that mimic legitimate websites. Fraudsters can replace them within hours to evade their detection. The effects of phishing attacks exhibit the need for anti-phishing mechanisms. Several approaches were there to recognize the phishing websites, the white list approach, blacklist approach, machine learning, and heuristic-based approach. Earlier studies have shown that classifiers may be subject to evasion attacks although this point has only been explored on a small scale. As a result, the study covers evasion attacks and their detection within the context of website classifiers, which is rarely explored. In response to the inadequacies, the proposed technique includes extracting information from URLs and classifying webpages using various machine learning methods. The methodology involves crafting adversarial samples targeting classification features, with a focus on maintaining the functionality and appearance of phishing websites. The appearance is evaluated using image distortion metrics named mean squared error. Then a resemblance approach is utilized for the aim of detecting assaults that happened as a result of evasion attacks. This research introduces a novel defense mechanism against evasion attacks, marking a significant contribution to the field.

INDEX TERMS Adversarial sample, DOM tree, evasion attack, phishing, similarity analysis.

I. INTRODUCTION

Phishing is now more dangerous to online security and in today's world, 50% of all emails include a bogus link that leads to fake sites. These websites may be fraudulent, in which attackers transmit phony emails or utilize fake websites which mimic the originals to get user credentials. Scammers employ various tactics to defraud people, including VOIP (Voice Over IP), fake websites, and covert redirection. The attacker develops counterfeit websites that are well-built and have a design that is comparable to real websites. The lack of awareness of

end-users of security policies and the demerits of weaker web servers are also exploited by phishers. To mislead end-users that the fraudulent website is authentic, attackers acquire web security certifications. According to government data, phishing attempts have been on the rise for the past half-century.

Severe regulations are prohibiting such acts, and those who break them face penalties such as fines or jail. The security forces should undertake instant action toward these types of breaches because the phony websites may just exist a few times, as well as the adversary, may vanish into cyberspace after engaging in unlawful acts. The anti-phishing legislation was enacted by the Indian government under Section 43 of the Computer Crime Act in 2000 [1] to prohibit

The associate editor coordinating the review of this manuscript and approving it for publication was Tyson Brooks³.

such violations. Even if the legislation can help to avoid phishing attempts to some level, it cannot eliminate them. Anti-phishing technologies and heuristic-based approaches thus have evolved to minimize phishing assaults. There are several methods for detecting and avoiding phishing scams, and it is necessary to accurately forecast the assault to protect the victims. The most difficult problem is that the counterfeit website appears to be real, making it difficult to tell if it is a phishing site or not.

Government legislation was enacted to combat phishing assaults, while anti-phishing technologies like blacklists, and machine learning-based systems were developed to identify phishing websites. The classifiers based on machine learning are excellent at finding those types of phishing sites still some of the classifiers are a risk of evasion attacks when set on the client-side. The evasion attack happens when the network is fed with adversarial examples and will generate as shown in Fig 1. To a human, it will seem just like its uncompromised copy means the adversary creates an adversarial example for assault, which manipulated copy. Because existing assaults in this area destroy the functionality and look of websites, this sort of danger receives minimal attention.

The methodology proposed shows the extraction of features from the URL and classification of websites. After that, the evasion attacks are done by crafting adversarial samples without spoiling the functionalities and appearances of phishing websites. Depending on the current phishing cases recognized by the implemented classifier, replicating the attacker's method to produce new perturbed samples. To see if the created instances can escape the classifier, the attacker produces new examples based on present phishing instances and assuming that the attacker has total control over the web page. It is achieved by producing an adverse sample out of a recognized phishing website. It contains information about the attacker's expertise and will continue till the classifier gets fooled more by a phishing attack like an authentic one. The assaults have three major advantages: effectiveness (100 percent attack success rate on specified classifiers), efficiency (samples constructed in the shortest time possible), and speed (samples crafted in the shortest time possible). Non-destructiveness means all adversarial samples created have the same look and functionality as their genuine counterparts.

The appearance can evaluate using image distortion metrics like mean squared error and mean absolute error. A similarity-based method is used for reducing the threat which will occur as a result of evasion attacks. Also, this is the first effective defense to work against evasion attacks in this area.

Prior research shows that machine learning-based classifiers are vulnerable to evasion attacks. But it is studied only in a little range when it comes to the anti-phishing domain rather than image recognition and malware detection. Unlike adversarial pictures, which must retain just the look, and adversarial malware, which must preserve just the functionality, hostile phishing websites must maintain

both appearance and functionality to achieve efficiency in web phishing [2]. Few works discuss the evasion attacks in web phishing which is an effective defense against these attacks. Accordingly we can understand whether it is possible to launch evasion attacks on machine learning based anti-phishing classifiers while preserving the functionalities and appearances of the web pages [3].

The main contributions of the paper are presented below:

- **Evasion Attack Focus:** Our study innovates by emphasizing the implementation of evasion attacks on web phishing classifiers, providing insight into attacker methods.
- **Adversarial Defense:** We introduce a novel defense mechanism that assesses the similarity of phishing and adversarial web pages, enhancing classifier resilience.
- **Dataset Limitation Acknowledgment:** We openly address dataset limitations, contributing to a more transparent assessment of its relevance.
- **Interdisciplinary and Practical:** Our research bridges cybersecurity, machine learning, and adversarial machine learning, offering practical solutions to enhance web user security.

The main objective is to implement evasion attacks on machine learning-based web phishing classifiers and to implement various phishing website classifiers to classify phishing and legitimate websites. Depending on the current phishing cases identified by the classifier, replicating the attacker's method produce a new perturbed sample [4]. The main objective of the paper is as follows:

- 1) **Evasion Attack Implementation:** Develop and implement evasion attacks on machine learning-based web phishing classifiers to assess their vulnerability to adversarial manipulation.
- 2) **Phishing Website Classification:** Create multiple phishing website classifiers to effectively differentiate between phishing and legitimate websites using machine learning techniques.
- 3) **Attacker's Method Replication:** Replicate the attacker's methodology by generating new perturbed samples based on the current phishing instances identified by the classifier.
- 4) **Defense Method Development:** Design and implement an effective defense mechanism aimed at reducing evasion attacks. This defense mechanism will check the similarity between phishing web pages and their corresponding adversarial pages, employing techniques such as mean squared error for evaluation.

Web phishing is a serious cyber security threat in the current situation. There are only a few works that concentrate on evasion attacks on machine learning-based classifiers. This is the first work that crafts the adversarial site using its phishing instance. Furthermore, the suggested defensive method is extremely successful against evasion assaults created by hostile samples. This research is useful in the investigation of evasion attacks. It also aids in the

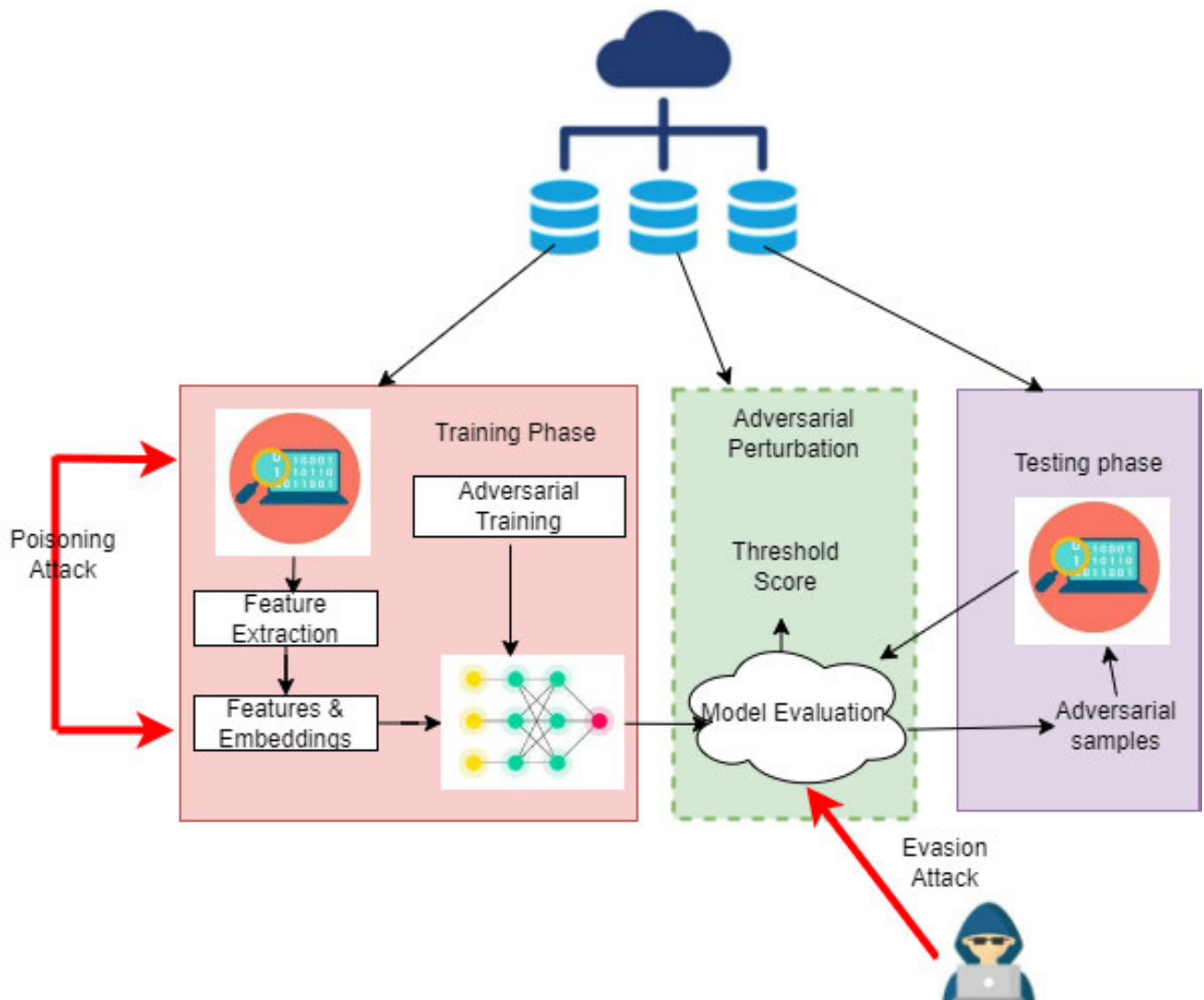


FIGURE 1. Block diagram of evasion attacks and defense mechanisms for web phishing classifiers.

identification of evasion assaults and the finding of hostile samples. Overall, it reduces the threat of web security.

The rest of the paper is structured as follows. Session II discusses phishing classification and adversarial methods, as well as other relevant topics. The techniques utilized in this investigation are described in Section III. The results are revealed in Section IV, and the conclusion and future work are presented in Section V.

II. RELATED WORKS

Several solutions come into effect for finding phishing websites in which most of them couldn't make decisions perfectly and accurately. This section includes the anti-phishing approaches and methods adopted in establishing solutions to decrease the false positive rate. Mainly there are white-list approaches, black-list approaches, and machine learning-based approaches in the classification method of

phishing websites. The machine learning-based approach that can detect phishing websites is at a high risk of evasion attacks caused by adversarial samples. So, this section also discusses adversarial samples and evasion attacks.

The method proposed in [5] by Jain suggested an evasion attack framework against a machine learning phishing URL detector. There is an algorithm for creating adversarial samples. Then recreated 41 machines learning-based phishing URL (MLPU) detectors and also compared their performance to the baseline. After that, they mimicked an evasion attack to test the machine learning phishing URL detector against the perturbed samples. The motivation behind this investigation is to uncover security weaknesses in MLPU solutions. The technique is broken into three categories: producing an adverse example, rebuilding and testing, retraining machine learning techniques, and simulating an evasion assault, according to the approach. Data preparation, site adversary,

path adversary, and TLD(Top Level Domain) adversary are all phases during the first stage, which is producing an adverse sample. To get meaningful URL tokens pre-processed for each URL. The URL is parsed using the python libraries urllib and tldextract to obtain some parts like IP address, port, sub-level domain, domain, TLD, path, and others. In the domain adversary, the algorithm changes the domain name of the URL to character and word. In path adversary, the algorithm changes the path of the URL. Test the pre-trained models against the created adversarial Phishing URL to mimic the evasion attack. The proportion of adversarial phishing URLs misclassified by baseline algorithms is used to determine the success rate of the attack. The research reveals MLPU system limitations. It also has a suggestion for further research into analysing and upgrading these systems.

Chen presented in [6] is a simple but effective method for imitating attacks. It uses direct feature manipulation to generate adversarial samples. At first, the paper discusses the attacker's aim, knowledge, and influence, and then explains them in the context of the phishing problem. Security violation, attack specificity, and error specificity are three separate aims for attackers. When coming to the attacker knowledge attacker may have complete knowledge of the model (white-box or perfect knowledge), little knowledge of the model (zero-knowledge), or restricted knowledge of the model (grey-box knowledge). Coming to the attackers influence two most important types are poisoning attacks and evasion attacks. According to the attacker's ability creating an adversarial sample from current phishing cases. After that determining the vulnerability level of the four datasets used.

The adversarial samples creation is based on the current phishing instances recognized by the classifier and imitating the attacker's method to produce new adversarial samples. To see whether the produced instances can avoid the classifier, the adversary produces new instances based on the present phishing instances.

First of all, the dataset was done and after that, the features were selected for manipulating the samples. Finally, the vulnerability level of each dataset is calculated. According to this study, phishing detection methods are sensitive to adversarial learning approaches. By altering a single characteristic, the detection rate for phishing websites was reduced to 70%. The recognition rate decreased to 0 percent after four characteristics changed. This indicates that each phishing sample that would have been identified successfully by a classifier model may be bypassed by altering only four feature variables. For each dataset, they established a vulnerability level that quantifies the number of characteristics that may be modified as well as the cost of such modification. It will be very helpful in the future for comparing various defensive models using such a metric. Overall, this work discusses the limitations in the case of machine learning algorithms when adversarial samples are taken into account. Also based on these evasion attacks established a concept of vulnerability level for the datasets [7]

Blacklist techniques are ineffective in detecting new phishing attacks and have a high percentage of false positives. Furthermore, previous machine learning-based approaches collect characteristics from third-party sources, such as a search engine. As a result, they are complex, sluggish, and unsuitable for real-time environments. Abaid [8] proposes a unique anti-phishing technique based on machine learning that exclusively collects characteristics from the client-side. The authors looked at all of the features of phishing and non-phishing websites and came up with a list of nineteen standout qualities that can be used to tell the difference between phishing and legitimate websites. For successful identification of a wide-ranging phishing assault, this method collects and analyses numerous features of suspicious websites. This paper's main contribution is the selection of an excellent feature set. Six additional elements were proposed to increase the detection accuracy of phishing web pages. Following feature extraction, researchers use heuristics to construct a feature vector for each website, resulting in a labeled dataset with a unique feature vector for each website. For statistical techniques in machine learning algorithms, the feature vector is a quantitative assessment of the feature. Each characteristic generates a value between 1 and 0, with 1 indicating fraudulent and 0 indicating genuine. A random forest (RF) classifier is developed in the training stage by using a feature vector extracted from each input in the training dataset. The classifier assesses not if a particular website seems to be a phishing site during the testing phase. This was the overall structure of the system architecture.

In feature extraction, there are eight URL-based features, one CSS feature, six hyperlink-specific features, three web identity features, and one login form feature. All these features are given in detail in the work. In the implementation details, they have shared in detail how they implemented this work. To identify phishing sites, researchers first selected important and helpful characteristics. After that, they built a dataset by extracting features from both genuine and fake websites. The random forest model was trained using the labeled dataset. The recommended anti-phishing solution is implemented on a computer with a Pentium i5 CPU running at 2.4 GHz and 4 GB RAM. The Python language has been used to implement this method. The collection of information from a webpage necessitates the use of several libraries. The collection of information from a webpage necessitates the use of several libraries. These libraries may be downloaded and extracted from the main websites or added separately using the pip package for Python. BeautifulSoup is one of the libraries that is used to extract information from HTML and XML documents. Evaluated each of the features that are extracted using several libraries as mentioned above. Finally, the authors have also highlighted the various advantages of this approach. So, this work represented a method in which URL, hyperlink, CSS, login form, and identity characteristics were employed to provide a unique technique for screening phishing websites from the client-side.

Corona [9] proposes the implementation of automatic classification of phishing pages. The concept and functionality of a robust machine learning classifier that is created to discover phishing websites is discussed. This classifier is used to constantly check Google's phishing blacklist. This classifier examines the URL and contents of thousands of sites every day to identify whether or not they are phishing. This technology sorts web pages provided by users and gathered via email spam detection and also extracts and analyses a lot of information about these pages to successfully detect a broad range of phishing pages. These attributes define the URL structure, the website's hosting, as well as the HTML content of the page as gathered by a crawler. Based on these characteristics, a logistic regression classifier determines if a page is phishing.

The classifier's classification model is created using an online training procedure. The training procedure makes use of characteristics gathered by the classification system over the previous three months and labelled according to our publicly available blacklist. The design goals of this work specify four things. The first is to reduce the amount of reported false positives, the classifier should emphasize accuracy over recall and check a huge number of pages in which the majority are not phishing. Owing to the lack of real phishing pages among the sites analysed the classifier's false positive rate will be exceedingly low. Second, the classifier must also have a good recall rate. Even if the classifier misses the majority of the fraudulent pages, it can be useful but, it is insufficient to replace a manual method.

Third, the classifier had to be able to cope with noisy training data and the classifier must have to pick up consistent patterns regardless of any previous misclassifications. Finally, the classification model must quickly evaluate several websites as the URL collecting system receives thousands of extra pages to evaluate every day. The workflow of the system is like the first step collects information from the page's URL. The second phase crawls the web page and gathers domain information. The third phase collects features from the page contents if the crawl is successful. The last step provides a score to the web page based on gathered characteristics, which represents the likelihood that the page is fraudulent. This method automatically adds the page to the blacklist issued by the serving system if the score is high enough. So, from this work, the researchers think that if they can ensure a better and timely blacklist, phishers will be forced to run at a loss and stop this form of Internet crime.

Xu [10] discusses an android malware classifier as a case study and gives a statistical examination of the effect of adversarial evasion attacks on numerous linear and non-linear classifiers. The work investigates the whole range of possible assaults based on the adversary's understanding of the classifier. The findings indicate that changing just a few characteristics of malicious applications is feasible to corrupt linear classifiers which are done with more experienced adversaries lowering the classifier's detection rate. The non-linear attack is freer from this type of attack

which is also mentioned in this study. The recommended assault plan's high-level concept is that each opponent takes certain measures to start an attack. The first is to duplicate the intended classifier, which builds a model by extracting a collection of attributes from training data and assigning feature weights that represent their importance in causing harmful classification.

The second method is to extract a collection of characteristics from a targeted app, which will then be changed to prevent detection. The app's top qualities that generate a detrimental classification are then identified using the model created. Also identifying the model's best benign features. Finally, update the app by deleting the top harmful feature and replacing them with the top benign characteristics. The work demonstrates a real-world scenario of changing a malicious app with the objective of avoiding detection and hence proves the suggested assaults are a genuine concern. They illustrate in practice an evasion strategy for a malicious application and indicate that the attacks they are looked into are feasible and pose a legitimate danger to mobile malware classifiers moreover based on the findings of this work, provide recommendations for building robust classifiers.

Weilin Xu proposes a case study on PDF malware classifiers. They present a generic approach for finding evasive options for target classifiers dynamically. They assume a comprehensive understanding of feature extraction rather than particular classification method. For this it requires classification score information on produced variants and a general understanding of the classifier's expected features. A prototype method that identifies variations can bypass structural feature-based PDF malware classifiers dynamically. This entails creating operators for probabilistic PDF modifications. An oracle for determining whether a produced variation retains maliciousness. Also, a selective mechanism for promoting promising variations during the dynamic development, and a fitness function for every targeted classifier. In testing the system's ability to evade two current PDF malware classifiers: PDFrater and Hidost, a classification model was developed specifically to withstand evasion attempts. In a test with 500 malware data, the algorithm finds evasive variants with a 100 % success rate against both classifiers.

The process begins with a sample that displays harmful behavior and is identified as such by the target classifier. The goal of this technique is to discover an evading sample that retains harmful behavior but is incorrectly classified as harmless by a target classifier. Perform random modifications on the harmful seed to start a collection of variants. After that, a target classifier and the oracle analyze each variation. The classifier is a black box that produces an integer of the likelihood of input data being malicious. A predetermined threshold is utilized to check whether it is harmful or benign. The oracle has been used to see if a particular item has any harmful characteristics. A successful evading sample is identified as harmless by a target classifier but determined to be harmful by the oracle. If no evasive samples are identified

in the community, a subset of the produced variations is chosen for the following generation using a fitness metric that reflects progress toward discovering an evasive sample. Mutation operators modify the chosen variations randomly to create the population's future generation. The procedure is repeated unless an evasive sample is discovered. Finally, it demonstrates that how the standard technique of constructing machine learning classifiers may fail in the face of persistent adversaries.

Liang [11] proposes a case study on googles phishing pages filter [12] which is a classifier used in chromium and also in chrome to detect phishing websites. The classifiers cracking is a novel attack technique that targets client-side classifiers and the researchers successfully studied the google filter for phishing pages, the GPPF classifier model using the approach and retrieved enough knowledge to be used for evasion attacks. by using the classification algorithm and scoring criteria.

There are two key phases in the cracking process [13]. The first is retrieving the GPPF(General Perception Pre-training Framework) classifier model from Chromium, and the second is decoding the model's hashed characteristics. Chrome/Chromium has a multi-process design that makes it even more resilient. The process which is dedicated for browsing can download an upgraded model from the server in a regular basis and each render process passes through an Inter Process Communication channel which enables categorization in the render phase. This would score the requested page to determine whether it is a phish or not. To retrieve encoded classification model information from Chromium, the debugging approach is used. Researchers found and confirmed the GPPF's scoring function `ComputeScore()`, which is a function of the `Scorer` class specified in the document scorer. cc, by statically analysing the source files of Chrome and analysing the rendering process of a phishing website with a valid one with the debugging tool gdb. They finish the classification workflow by merging a dynamically backward tracing of the execution path that began with `ComputeScore()` as well as a static examination of the program code.

The classifier retrieves three types of page characteristics from the present web page: URL, DOM(Document Object Model), and Term features, in that sequence. Then it gathers page characteristics by using a hashing mechanism SHA-256 technique and is given to the method `ComputeRuleScore()`, which uses the scrambled model features to calculate the rule score by each scoring rule. Finally, `ComputeScore()` adds all of the rule scores together to provide a final score for the current page. Finally, the score is evaluated to a set of criteria (say 0.5). Researchers must retrieve the weights and model characteristics for each scoring rule to break the classifier. Analysing the Render process will reveal the rule weight. They set a halt within `ComputeRuleScore()`, where a gdb script extractor is used to retrieve the weight information out from rule entities in storage and write this to

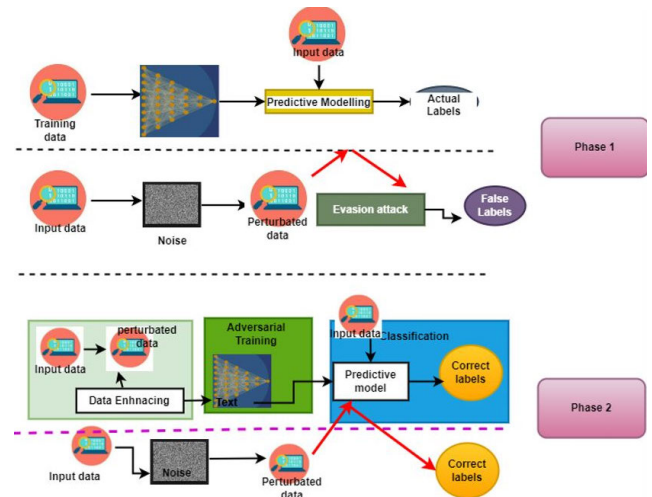


FIGURE 2. Framework of proposed system.

a file. To decode the hash model characteristics, users must first understand their meaning and how they translated into the page characteristics. Three types of page characteristics will be translated into a matching model associated with different methods when calculating the score and gathered four datasets linked to URLs to conduct focused brute force assaults to discover as many potential collisions as feasible. This research showed an essential fact that client-side classification methods have wider security risks and, as a result, are more vulnerable to assaults [14].

III. METHODOLOGY

The data collection is the first step to be performed then the implementation of classifier [15] is done. Following that, adversarial samples [16] are created, and attacks were evaluated. Finally, a detection method is introduced to find the evasive samples in which the DOM tree similarity is calculated to identify the evasive samples. The Fig 2 represents the framework.

A. DATACOLLECTION

In this study, we employed a dataset collected from two primary sources, namely Phishtank and the Alexa dataset, to investigate evasion attacks on machine learning-based web phishing classifiers. The dataset is integral to our research as it forms the foundation for both attack implementation and defense mechanisms. These sources were chosen because they represent real-world data and are widely recognized in the field of web security.

1) PHISHTANK DATASET

- Phishtank [17] is a widely recognized and reputable resource for phishing data in the cybersecurity community. It crowdsources phishing URLs submitted by users and verifies their legitimacy.

From Phishtank, we extracted a total of 5,000 phishing URLs. These URLs were selected based on their

historical records of being associated with phishing activities.

2) ALEXA DATASET

- The Alexa dataset [18], while renowned for providing information about website rankings and domains, offers domain-level data without the path segments of specific web pages.
- We collected 5,000 legitimate URLs from the Alexa dataset. These URLs were chosen for their representation of well-established, non-malicious websites.

we consider the “evasion attacks” as the strategies and techniques that phishing websites might use to avoid detection by security systems. These evasion tactics can include but are not limited to obfuscation, redirection, and content manipulation. The dataset does not consist of actual “evasion attacks” as separate entities; instead, it consists of URLs that may employ such evasion tactics to avoid being classified as phishing.

In this dataset, each URL is associated with a label that signifies its nature. For labeling, we assigned a value of -1 to phishing URLs and 1 to legitimate URLs, making this dataset suitable for binary classification tasks. The labels “phishing” and “legitimate” are essential for training and evaluating the performance of our classifiers. This binary labeling scheme is used to distinguish between phishing and non-phishing URLs for the purpose of training and evaluating the classifier’s performance [19], [20].

B. FEATURE EXTRACTION

We extracted a set of 15 features from these URLs. These features were carefully chosen based on their relevance to phishing detection and have been established as effective indicators for classifying phishing websites.

The selected features encompass various aspects of URLs, such as domain-based, content-based, and lexical characteristics. These features include but are not limited to domain entropy, URL length, presence of special characters, and the number of subdomains. The location of a particular unique page on the Internet is known as a URL. Each correct URL, on principle, refers to a single resource. A Webpage, a CSS sheet, a picture, and so on are examples of these resources. There are very few exclusions in practice, the most frequent of them is a URL linking to a resource that no longer holds or has relocated. The features collected out of a URL are used to identify whether or not the URL is phishing. The quality of features supplied into the model is critical to the success of any learning endeavor. The feature extraction rules are used here which extracts the features based on the rules given. All 15 features are extracted from the URL dataset. The last dataset contains the feature retrieved information and also the class labels.

C. REBUILDING CLASSIFIER

Using the extracted features built some classifiers to classify phishing and legitimate sites [21]. After the classification,

selecting a phishing site and crafting its adversarial sample. K-nearest neighbour, logistic regression, support vector machine, decision tree and random forest are the classifiers [22], [23] that are implemented as a part of this. The classifiers will take the extracted features as input and outputs the accuracy. The best classifier is chosen according to the accuracy. After that, the user can give a test input URL and the best classifier will predict that the sample given belongs to which category.

D. CRAFTING ADVERSARIAL SAMPLES

The new perturbed samples are produced depending on the current phishing cases recognized by the classifier, replicating the attacker’s method. Also, to see whether the produced instances can escape the classifier, produced a new example based on the present phishing instance and assuming that the attacker has total control over the web page. After identifying the adversarial website is building using web app in angular. Researchers anticipated that every attribute might be substitute by attributes found in a previous phishing attempt. To generate a new instance, first studied the phishing webpage and then build the adversarial site for the same according to the features of the same.

E. EVALUATION OF ATTACKS

The test pages are checked using the target classifier. That is give the URL as an input and the classifier will classify the adversarial site as legitimate. For further evaluation comparison of phishing and the adversarial screenshot is done by using the criteria mean squared error. There will be two images one is of a phishing site and its adversarial site. The mean squared error should be calculated to identify the similarity in appearance which means to check whether the adversarial sample is the same by appearance or not. All of the produced adversarial instances have always functioned and seemed identical to their original counterparts, with appearance determined by a measure of mean squared error (MSE), a typical picture distortion criterion. Since the distortion metric is a criterion that provides a quality number to the picture, and when it displays zero, it indicates that two images are identical, the MSE value will be zero, demonstrating that the attacks retain appearance.

The MSE can be calculated by:

$$MSE = \frac{1}{mn} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} [I(i, j) - K(i, j)]^2 \quad (1)$$

The following section demonstrates the variation in pixels values between both photos, which would be squared and termed as squared error, whereby m and n seem to be the aspect ratio of the screenshots. MSE is simple to install, but it may have difficulties when it comes to similarity. The major one is that big differences in pixel values may not always imply that the contents of the picture are drastically different. An MSE value zero indicates perfect similarity. MSE 1 indicates less similarity and the that number will

continue to climb as the mean gap between pixel values increases.

F. DETECTION OF EVASION ATTACKS

The detection module [24] detects evasive pages by comparing the similarities of nodes. The main aim is to evaluate two DOM trees obtained from the phishing and adversarial parsed HTML files, then use an already build algorithm to determine how similar they are. How can two trees with different nodes be compared? Thus, in addition to processing an Html page and retrieving a DOM tree, the real step that completed is to create a method for comparing two tree structures with different nodes and determining their resemblance level. A property of a tree structure, such as the periodic differences for nodes holding the very same features, should be concentrated to offer the comparison method and determine how equivalent they are with each other. As a result, this module must be structured as a tree: classes defining node, procedure or method to traverse over a tree, techniques to retrieve the functions and functionality of every node, and specific methods to perform comparison and resemblance assessment. The Java application is created as part of this module and it may be simply run out of a simple JAR executable. Eclipse IDE was picked as the integration platform for this language choice. tree similarity is the description of this module that has settled in the Eclipse IDE environment. There is a bundle for the entire project in the src special folder trees, that can be integrated into a framework. All Java scripts are included in this bundle. Each one has the .java file format and includes the essential classes. The treeSim.java seems to be the primary file from which the application is launched following compilation and the fundamental method is located. The remaining classes are attribute class, element class, and Text class which defines node kinds. The core java language's idea is built upon objectoriented coding; also, the primary procedure including its code is contained inside a class. Dynamically typed coding has been effectively used for the development of the remedy to the examined problem due to its advantageous qualities. The trees module makes up the significant phase of this work. This method imports various classes which describe the component kinds. The element node type inside the Modelling of class provides element nodes. It contains the following information: The name of an entity, as well as its periodicity, which shows how many times that component appears in the tree. The attribute node type represents the feature nodes. The text node class defines a text node. Treesim class, which is at the center of the notion, encapsulates the fundamental approach. It comprises practically any of the remaining operations inside the program, as well as the scripts for creating the application's user interface, including the wrapping of the main function. The above-mentioned 3 types are loaded into the treesim class.

Two HTML websites are compared using an algorithm. Following the collection of the relevant information, comparison methods based on node kinds are used. Element nodes

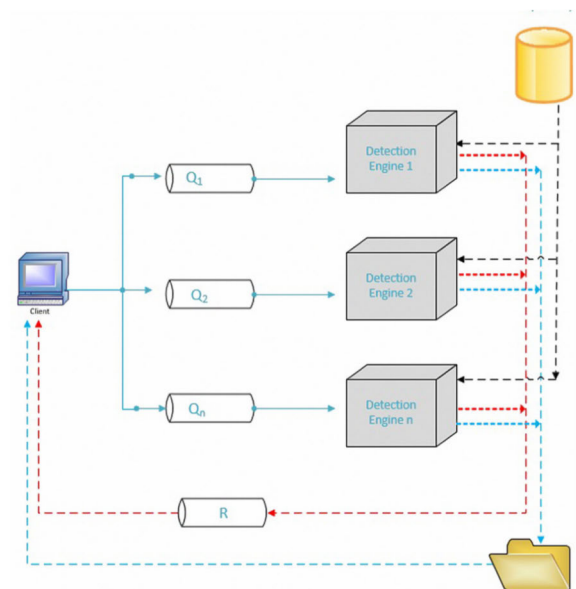


FIGURE 3. Detection module.

are the knowledge and opportunities for all different parts in the iterated HTML document and the frequency components for all separate items for two trees, least frequency by each component is included in a specific frequency list for common characteristics. The total can be divided by a large number among element nodes for each tree. The resemblance ratio of the element node produces by multiplying the outcome by 100. The same method which is used for element nodes can be used for attribute nodes too. This phase results in a quantitative relation. Also, the same procedure can perform on element nodes with only one feature. This step yields yet another ratio. The ultimate resemblance ratio regarding the attribute nodes was calculated by averaging these results. The same procedure is used with text nodes and the resemblance ratio is calculated. The sequence of influence might be element nodes and should be greater than text nodes and attribute nodes. Here the 60% is allocated for element nodes, 30% for text nodes, and 10% for attribute nodes. The total resemblance ratio among the dom trees gets calculated using this assumption. The overall working of the algorithm proposed in the module is shown in Figure 3.

The composition of an element node class is represented by this class. It contains the component name as well as its frequencies. After that, the layout of an attribute node is represented by the class. It contains the attribute's name as well as its frequency. Next is the text node class and it contains the name of the parent node which owns the textual node, as well as the frequencies of such a node. Finally, the main one is treesim class which is the most important one and which includes the main method. Most of the functions were built here, including the main method. After this method's compilation, the program is launched. The class's predefined function Object is treeSim(). This function Object contains

TABLE 1. Sample dataset from phishtank.

	URL
0	https://online-unadd-device.com/
1	https://lopekibd.com/
2	https://www.chase.megjob.com/
3	https://fb-item27818931marketplace.com/
4	http://hsbc.authenticate-webpersonal.com/
5	https://royalmail.co.uk-trackparcel.com/
::	::::::::::::::::::::::
::	::::::::::::::::::::::
::	::::::::::::::::::::::
998	https://anarosha.ltd/template/adupte/index.html
999	http://rulesfbw7c7p88m8gyp.webport.ca/connect.html
1000	http://mihchigini.club/css/

instructions for loading GUI components as well as adding interaction triggers.

IV. RESULTS AND DISCUSSION

A. DATACOLLECTION

For the proposed method the experiments are done in a python programming language using a command prompt. The similarity is measured using scikit-image, or skimage package. Finally, a java application is rebuilt for checking the DOM node similarity which is used as a detection method of evasion attacks. Eclipse IDE was picked as the integration platform for this purpose. This module includes the creation of a Java programme that may be executed from a simple JAR executable. Here used two datasets Phishtank and Alexa dataset which are URL datasets. The sample set of data is given in Table 1 and Table 2.

A set of 10000 samples were collected, and 15 characteristics were derived from these URLs. These are elements that give some guidelines for detecting phishing websites. The dataset includes values for characteristics like phishing (−1), genuine (1), as well as the labels phishing (−1) and not phishing (1).

B. FEATURE EXTRACTION

The URL dataset may be used to extract a variety of characteristics. In this case, fifteen characteristics were retrieved from the dataset shown in Table 3. These characteristics may be utilised to create a classifier that distinguishes between phishing and authentic samples. Table 4 shows the final feature extraction data as well as the class labels.

C. EVALUATION OF CLASSIFIERS

The extracted features are used to build the classifiers which will classify the websites as phishing and legitimate. The observation of data before splitting i.e., number of websites that are marked as phishing and legitimate are given in Table 5. Each of these features are explained detail in methodology part. The 70% of the dataset will be used for training and 30% is used for testing i.e., in-case of this dataset 7000 samples are taken as training samples then 3000 taken as testing samples. There are five classifiers which

```
Enter a url
http://pmedulrn.com/mailwizz/customer/index.php/guest/index
-1
Phishing site
```

FIGURE 4. Output of phishing sample.

```
Enter a url
https://mailwizz-2ccb2.web.app/
1
Not a phishing site
```

FIGURE 5. Output of crafted sample.

are implemented. While implementing the classifiers such as k-nearest neighbor, logistic regression, SVM, decision tree and random forest the accuracy of each classifier is given in the Table 6.

The accuracy of logistic regression is the smallest when it comes to test data. The accuracy of random forest is the highest among all. According to these models, 1493 samples are phishing websites and 1507 samples are legitimate from the test set. The precision, recall, f1-score, and support of all classifiers are obtained.

The precision is derived by dividing the true positives by the total of true positives and false positives, and it represents the proportion of the results that are meaningful. The recall will be the proportion of relevant results that are properly categorised, divide the true positives by the sum of true positives and false negatives to get the true positives/false negatives ratio. The number of true positives, true negatives, false negatives and false positives are TP, TN, FN and FP respectively. The f1 score will be the weighted average of precision and recall and the support will be the actual number of occurrences of the class in the given dataset. The mathematical representation of these metrics is shown below:

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

$$F1 - score = 2 * \frac{precision * recall}{precision + recall} \quad (4)$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (5)$$

The confusion matrix of all classification methods is generated. The comparison of the accuracies of given classification methods are shown in following Tables 7, 8, 9, 10 and 11

From the table it is clear that the random forest algorithm has high accuracy in comparison with K nearest neighbor, logistic regression, support vector machine, decision tree.

D. RESULTS OF CRAFTING ADVERSARIAL SAMPLES

It is a very tough process to craft adversarial samples using the phishing instances and the method here used is to imitate phishing websites and checking whether the sample evade the classifier. With the subset of features identified it is a hectic

TABLE 2. Sample dataset from Alexa.

	URL
0	http://1337x.to/torrent/1048648/American-Sniper-2014-MDiTAlIAN- DVDSCR-X264-BST-MT/
1	http://1337x.to/torrent/1110018/Blackhat-2015-RUSSIAN- 720p-WEB-DL-DD5-1-H264-RUGFT/
2	http://1337x.to/torrent/1122940/Blackhat-2015-x264-1080p- WEB-DL-eng-nl-sub5-sharky/
3	http://1337x.to/torrent/1124395/Fast-and-Furious-7-2015- HD-TS-XVID-AC3-HQ-Hive-CM8/
4	http://1337x.to/torrent/1145504/Avengers-Age-of-Ultron 2015-CAM-New-Audio-x264-CPG/
5	http://1337x.to/torrent/1160078/Avengers-age-of-Ultron- 2015-HQ-CAM-H264-AC3-MURD3R/
::	
::	
::	
998	http://1337x.to/torrent/1058353/The-Theory-of-Everything- 2014-1080p-BrRip-x264-YIFY/
999	http://1337x.to/torrent/1110014/Blackhat-2015-RUSSIAN- 1080p-WEB-DL-DD5-1-H264-RUGFT/
1000	http://1337x.to/torrent/1123461/Hot-Tub-Time-Machine-2- 2015-UNRATED-HDRip-XViD-ETRG/

TABLE 3. Extracted features.

Index	Features
0	Having IP
1	URL Length
2	Having At
3	Redirection
4	https Domain
5	Tiny URL
6	Prefix/Suffix
7	DNS Record
8	Web Traffic
9	Domain Age
10	Domain End
11	iFrame
12	Mouse Over
13	Right Click
14	Web Forwards

TABLE 6. Comparison of different ML based method in terms of accuracy

Algorithm	Accuracy
K-nearest neighbour	74.8 %
Logistic regression	87.63 %
Support vector machine	87.63 %
Decision tree	88.03 %
Random forest	88.06 %

TABLE 7. Classification report of KNN.

Precision	Recall	F1-Score	Support	
0.70	0.85	0.77	1493	-1
0.81	0.65	0.72	1507	1
-	-	0.75	3000	Accuracy
0.76	0.75	0.75	3000	Macro avg
0.76	0.75	0.75	3000	Weighted Avg

TABLE 4. Features extracted from URL.

0	1	1	1	1		-1	-1
1	1	1	1	1		-1	-1
2	1	-1	1	1		-1	-1
3	1	-1	1	1		-1	-1
4	1	1	1	1		-1	-1
5	1	-1	1	1		-1	-1
6	1	1	1	1		-1	-1
7	1	-1	1	1		-1	-1
8	1	1	1	1		-1	-1

TABLE 5. Observation of data.

Class	Observations
1	6157
-1	4898

ask to create a sample so created samples using this method. The output of the same is given in the Figure 4 and Figure 5.

Crafted the sample by using the phishing instance and webapp in angular and the sample correctly evades the classifier. The adversarial sample is launched using firebase. The two outputs show the same websites one is phishing sample and other is an adversarial sample. This is the first time an adversarial sample is crafted using phishing instance and the classifier doesn't recognize these samples. The random forest made a good accuracy in the case of

test data so we only take the random forest classifier for evaluation [25], [26].

Our method operates in a white-box scenario, where the attacker has knowledge of the classifier’s architecture and may aim to manipulate it for malicious purposes. The attacker’s goal is to craft adversarial examples that evade the classifier’s detection by exploiting its vulnerabilities. Our defense mechanism assures confidence and similarity constraints by evaluating the appearance and content of web pages, thereby providing a robust defense against adversarial manipulation. Our technique is designed to address both targeted and untargeted attacks, considering the multifaceted nature of evasion tactics. The threat model encompasses a range of evasion strategies, including obfuscation, redirection, and content manipulation, which are commonly employed by adversaries to trick web phishing classifiers [27]. By addressing these aspects, our research contributes to a comprehensive understanding of adversarial machine learning in the context of web phishing detection and defense.

E. EVALUATION OF ATTACK

The evaluation of the attacks is carried out by checking the mean squared error of the phishing and adversarial pages. Fig 6 shows two images: the first is a screenshot of a phishing sample, and the second is a snapshot of an adversarial sample.

TABLE 8. Classification report of logistic regression.

MSE: 0.00

Precision	Recall	F1-Score	Support	
0.98	0.77	0.86	1493	-1
0.81	0.98	0.89	1507	1
-	-	0.88	3000	Accuracy
0.89	0.88	0.87	3000	Macro avg
0.89	0.88	0.87	3000	Weighted Avg

TABLE 9. Classification report of SVM.

Precision	Recall	F1-Score	Support	
0.98	0.77	0.86	1493	-1
0.81	0.98	0.89	1507	1
-	-	0.88	3000	Accuracy
0.89	0.88	0.87	3000	Macro avg
0.89	0.88	0.87	3000	Weighted Avg

TABLE 10. Classification report of decision tree.

Precision	Recall	F1-Score	Support	
0.99	0.77	0.86	1493	-1
0.81	0.99	0.89	1507	1
-	-	0.88	3000	Accuracy
0.90	0.88	0.88	3000	Macro avg
0.90	0.88	0.88	3000	Weighted Avg

TABLE 11. Classification report of random forest.

Precision	Recall	F1-Score	Support	
0.99	0.77	0.87	1493	-1
0.81	0.99	0.89	1507	1
-	-	0.88	3000	Accuracy
0.90	0.88	0.88	3000	Macro avg
0.90	0.88	0.88	3000	Weighted Avg

of the same site. It will take only four lines of code to calculate the mean squared error. The mean squared error function will accept inputs such as image A and image B, i.e., the two images to be compared. The pictures were then converted from unsigned 8-bit integers to floating point, and the difference between them was calculated by subtracting the pixel intensities and squared. Then the mean of the MSE is calculated by dividing sum of squares by the total number of pixels in the image and finally returning MSE to the caller one. It is important that the value of MSE is 0 if perfect similarity is provided in the appearance and functionality. The value greater than one implies less similarity.

F. RESULTS OF DETECTION MODULE

In this section, we provide an overview of our detection module, which is a key component of our methodology for web phishing classification and evasion attack defense. The detection module is designed to assess the similarity between web pages and their corresponding adversarial versions, ultimately serving as a safeguard against evasion tactics. However, we recognize that the original content lacked scientific arguments, and we have taken steps to improve its clarity and scientific basis [28]. In the detection module finding the evasive page using the similarity of DOM



FIGURE 6. Evaluation of attack using MSE.

nodes. It will check the element similarity, text similarity and attribute similarity. Finally outputs an overall similarity. If the overall similarity is higher then, the two sites are similar according to the DOM node similarity which means the legitimate sample given is an adversarial one. This is a case of evasion attack. If a phishing page and its adversarial page is given then it will show higher similarity. For finding similarity between designs of two webpages leads the similarity between their HTML files. The similarity between these HTML files leads the similarity between their DOM trees. So, evaluation of DOM tree similarity is a solution to the main problem of the webpage similarities. An algorithm is used for measuring the similarity level between DOM trees which is extracted from two HTML files which is already developed.

The main objective of this model is to develop a system that measures the similarity between 2 webpages with respect to designs. So, this system can be used to check whether adversarial and phishing site has higher similarity. There are various steps when coming to the evaluation the first step is parsing the webpages which is loaded to the system. Then the extraction of DOM object with their relatives. An algorithm is used to compare these DOM trees and calculate the similarity. The similarity is given as the output of the DOM similarity evaluator [29]. The Fig 7 and fig 8 shows the user interface of DOM similarity evaluator which includes buttons for loading the webpages then it shows each type of similarity and finally the overall similarity. There also a button called quit the application to quit the process. The next one in Figure 8 shows the calculation of similarity of phishing page and the crafted page it shows 70 percentage of overall similarity. This means both the pages are somewhat similar and the given webpages are phishing and its own adversarial instance [30].

V. CASE STUDY

In this study, we focus on tackling evasion attacks in anti-phishing methods. We build on established approaches like white lists, blacklists, and machine learning for spotting phishing websites. Our new method uses machine learning to analyze URLs and categorize webpages. What makes our study unique is our investigation into evasion attacks within website classifiers. We create tricky samples that aim at

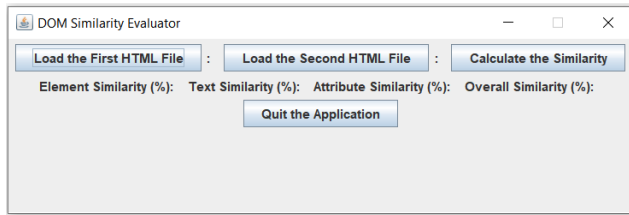


FIGURE 7. User interface of DOM similarity evaluator.

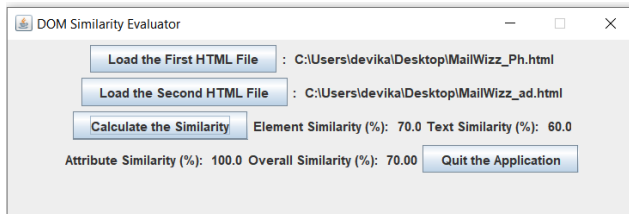


FIGURE 8. DOM similarity evaluation output.

classification features, keeping phishing websites' functions and looks unchanged. Using metrics like mean squared error, we measure how well evasion attacks work. In response to feedback, we connect our work to online hackers, highlighting the practical impact of countering evasion attacks used by hackers to exploit security vulnerabilities. This study stands out by introducing an innovative defense against evasion attacks, filling a crucial gap in current anti-phishing strategies and offering practical insights for cybersecurity in the ever-changing online landscape.

In this study, we underscore the practical relevance of our research by providing tangible examples of its applications in addressing real-world challenges. Consider a scenario where a financial institution utilizes our proposed machine learning-based phishing detection system. Traditional methods may struggle to keep pace with the evolving tactics of online hackers who continuously refine their evasion techniques. Our approach, however, actively identifies and defends against these evasion attacks, safeguarding users' sensitive information.

Furthermore, imagine an e-commerce platform implementing our methodology to protect customers from fraudulent websites. By crafting adversarial samples that mirror sophisticated phishing attempts, we simulate real-world scenarios, demonstrating the robustness of our defense mechanism in preserving the appearance and functionality of genuine websites while thwarting malicious activities. These practical examples showcase the direct applicability and effectiveness of our research in enhancing cybersecurity measures across diverse online domains.

VI. CONCLUSION

This work mainly discusses about the evasion attacks and its detection in the case of websites. The analysis of URL dataset is carried out and the features that are important is extracted from the dataset. Using these features the classification

models such as K-Nearest Neighbour, Logistic regression, support vector machine, decision tree and random forest classifier was built. While comparing the test accuracy of all the models the decision tree gives the higher accuracy so it is taken for identifying the crafted sample. The sample is created by adding the legitimate nodes to the phishing site and it successfully evaded the classifier. The evaluation of the attack is done by using mean squared error and finally the detection is done using the DOM similarity evaluator. Rather than phishing websites, the adversarial websites which can act as legitimate websites will be more effective in web phishing. The evasion attack comes in to the category of adversarial attacks and tried to craft this attack but it is not a complete success. In the future work, a study of crafting adversarial samples in an easy way can be carried out.

VII. FUTURESOCPE

Looking ahead, the research could explore making it easier to create adversarial samples that mimic legitimate websites more effectively, improving our understanding of challenges in cybersecurity. This might involve finding better ways to create samples that can evade classifiers with a higher success rate. Investigating how evasion attacks might change over time could also provide useful insights for staying on top of new threats. Additionally, looking into using new technologies like deep learning or reinforcement learning in anti-phishing efforts could make defense systems stronger against advanced evasion tactics. This approach aims to be proactive in addressing potential advancements in how adversaries try to evade detection, constantly strengthening the ability of website classifiers to handle evolving cyber threats.

REFERENCES

- [1] F. Song, Y. Lei, S. Chen, L. Fan, and Y. Liu, "Advanced evasion attacks and mitigations on practical ML-based phishing website classifiers," *Int. J. Intell. Syst.*, vol. 36, no. 9, pp. 5210–5240, Sep. 2021.
- [2] B. Sabir, M. A. Babar, and R. Gaire, "An evasion attack against ML-based phishing URL detectors," *Tech. Rep.*, 2020.
- [3] H. Shirazi, B. Bezawada, I. Ray, and C. Anderson, "Adversarial sampling attacks against phishing detection," in *Proc. IFIP Annu. Conf. Data Appl. Secur.* Cham, Switzerland: Springer, Jul. 2019, pp. 83–101.
- [4] S. Anupam and A. K. Kar, "Phishing website detection using support vector machines and nature-inspired optimization algorithms," *Telecommun. Syst.*, vol. 76, no. 1, pp. 17–32, Jan. 2021.
- [5] A. K. Jain and B. B. Gupta, "Towards detection of phishing websites on client-side using machine learning based approach," *Telecommun. Syst.*, vol. 68, no. 4, pp. 687–700, Aug. 2018.
- [6] S. Chen, M. Xue, L. Fan, S. Hao, L. Xu, H. Zhu, and B. Li, "Automated poisoning attacks and defenses in malware detection systems: An adversarial machine learning approach," *Comput. Secur.*, vol. 73, pp. 326–344, Mar. 2018.
- [7] I. Vayansky and S. Kumar, "Phishing challenges and solutions," *Comput. Fraud Secur.*, vol. 2018, no. 1, pp. 15–20, Jan. 2018.
- [8] Z. Abaid, M. A. Kaafar, and S. Jha, "Quantifying the impact of adversarial evasion attacks on machine learning based Android malware classifiers," in *Proc. IEEE 16th Int. Symp. Netw. Comput. Appl. (NCA)*, Oct. 2017, pp. 1–10.
- [9] I. Corona, B. Biggio, M. Contini, L. Piras, R. Corda, M. Mereu, G. Mureddu, D. Ariu, and F. Roli, "DeltaPhish: Detecting phishing webpages in compromised websites," in *Proc. Eur. Symp. Res. Comput. Secur.* Cham, Switzerland: Springer, Sep. 2017, pp. 370–388.

- [10] W. Xu, Y. Qi, and D. Evans, "Automatically evading classifiers," in *Proc. Netw. Distrib. Syst. Symp.*, vol. 10, Feb. 2016, pp. 1–15.
- [11] B. Liang, M. Su, W. You, W. Shi, and G. Yang, "Cracking classifiers for evasion: A case study on the Google's phishing pages filter," in *Proc. 25th Int. Conf. World Wide Web*, Apr. 2016, pp. 345–356.
- [12] G. Harinahalli Lokesh and G. BoreGowda, "Phishing website detection based on effective machine learning approach," *J. Cyber Secur. Technol.*, vol. 5, no. 1, pp. 1–14, Jan. 2021.
- [13] A. Odeh, I. Keshta, and E. Abdelfattah, "Machine learning techniques for detection of website phishing: A review for promises and challenges," in *Proc. IEEE 11th Annu. Comput. Commun. Workshop Conf. (CCWC)*, Jan. 2021, pp. 813–818.
- [14] W. Zhuang, Q. Jiang, and T. Xiong, "An intelligent anti-phishing strategy model for phishing website detection," in *Proc. 32nd Int. Conf. Distrib. Comput. Syst. Workshops*, Jun. 2012, pp. 51–56.
- [15] P. Yang, G. Zhao, and P. Zeng, "Phishing website detection based on multidimensional features driven by deep learning," *IEEE Access*, vol. 7, pp. 15196–15209, 2019.
- [16] M. Aburrous, M. A. Hossain, K. Dahal, and F. Thabtah, "Predicting phishing websites using classification mining techniques with experimental case studies," in *Proc. 7th Int. Conf. Inf. Technol., New Generat.*, Apr. 2010, pp. 176–181.
- [17] M. Aljabri and S. Mirza, "Phishing attacks detection using machine learning and deep learning models," in *Proc. 7th Int. Conf. Data Sci. Mach. Learn. Appl. (CDMA)*, vol. 5, Mar. 2022, pp. 175–180.
- [18] A. Smith and B. Johnson, "Evasion attacks and their impact on machine learning-based phishing detection," *J. Cybersecur. Res.*, 2022.
- [19] X. Wang, Y. Li, and L. Zhang, "Evasion attacks on phishing URL detectors: A comparative study," *Proc. ACM Hum.-Comput. Interact.*, 2022.
- [20] S. Gupta, R. Singh, and R. Sharma, "A survey on adversarial attacks in machine learning for phishing detection," *Int. J. Inf. Manag.*, 2022.
- [21] S. Chaudhary and A. Verma, "Defending machine learning-based phishing URL detection against evasion attacks," *J. Cybersecur. Inf. Assurance*, 2022.
- [22] J. Wu, Q. Li, and Z. Liu, "Robust phishing detection using adversarial training," *IEEE Trans. Netw. Service Manag.*, 2022.
- [23] T. Zhang, H. Zheng, and Q. Zhao, "Enhancing phishing detection robustness through adversarial examples," *Secur. Commun. Netw.*, 2022.
- [24] S. Sundararajan, J. Rao, and V. Srinivasan, "Deep learning for phishing detection: Challenges and opportunities," *J. Inf. Secur. Appl.*, 2022.
- [25] V. Kumar and A. Sharma, "Evasion attacks on machine learning-based phishing URL detection: A comprehensive analysis," *IEEE Trans. Emerg. Topics Comput.*, 2022.
- [26] S. Raj and A. Das, "Machine learning and deep learning-based approaches for phishing detection: A comprehensive review," *J. Comput. Virol. Hacking Techn.*, 2022.
- [27] A. Chatterjee, R. Gupta, and A. Khan, "Understanding adversarial attacks in phishing detection: A deep learning perspective," *J. Comput. Syst. Sci.*, 2022.
- [28] C. Li, H. Wang, and Y. Liu, "Exploring evasion attacks and countermeasures in phishing detection using transformers," *J. Cybersecur.*, 2022.
- [29] W. Zhou, L. Chen, and L. Yang, "Machine learning models for phishing detection: A review and analysis of adversarial attacks," *J. Inf. Secur. Cybercrimes*, 2022.
- [30] N. Patel, S. Gupta, and P. Choudhury, "Mitigating evasion attacks in phishing detection using feature engineering and ensemble methods," *Int. J. Netw. Secur.*, 2022.



MANU J. PILLAI received the Ph.D. degree in computer science and engineering from the National Institute of Technology, Calicut. He is currently an Associate Professor with the Department of Computer Science and Engineering, Thangal Kunju Musaliar College of Engineering (TKMCE), Kerala, India. His research interests include wireless networks, deep learning, and smart environments.



S. REMYA received the Ph.D. degree in computer science and engineering from the Vellore Institute of Technology, Vellore Campus. She is currently an Assistant Professor with the Department of Computer Science and Engineering, School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri Campus, Kollam, Kerala, India. Her research interests include deep learning, data science, computer vision, and smart environments.



V. DEVIKA received the Bachelor of Technology degree from CAPE and the Master of Technology (M.Tech.) degree from the TKM College of Engineering, Kollam, with a distinguished academic background. Her research pursuits center around the domains of cybersecurity, with a particular focus on the identification of phishing websites and the detection of evasion attacks. Demonstrating a keen interest in advancing the understanding and mitigation of cybersecurity threats, she has contributed significantly to the evolving landscape of digital security.



SOMULA RAMASUBBAREDDY received the Ph.D. degree in computer science and engineering from VIT University, Vellore, India, in 2022. He is currently a Postdoctoral Researcher with the Department of Information and Communication, Sunchon National University, South Korea. He has more than 40 publications in reputed journals and conferences papers. His research interests include mobile cloud computing, the IoT, machine learning, and edge computing.



YONGYUN CHO received the Ph.D. degree in computer engineering from Soongsil University. He is currently a Professor with the Department of Information and Communication Engineering, Sunchon National University. His main research interests include system software, embedded software, and ubiquitous computing.

...